# Research

Ursache Ana-Maria + Repede Monica

May 29, 2026

## Coloring Big Graphs With AlphaGoZero

**Link:** `https://arxiv.org/pdf/1902.10162`

**Summary:** This project adapts the AlphaGoZero reinforcement learning algorithm by integrating it with graph embeddings to learn fast and effective heuristics for graph coloring entirely from scratch

**Solution & Method:** The authors formulate graph coloring as a Markov Decision Process (MDP). The method utilizes deep reinforcement learning alongside Monte Carlo Tree Search (MCTS) with Upper Confidence Bound (UCB) and self-play. To accommodate varied graph sizes, they introduce FastColorNet, a deep neural network architecture that computes a dynamically sized softmax assignment to predict the best coloring strategies by accessing the full global graph context

**Benchmarks:** The models were trained on synthetic random graphs and benchmarked on real-world graphs sourced from the publicly available SuiteSparse matrix collection. The sources do not specify a download link for their proprietary training framework.

**Other Info:** AlphaGo Zero is a groundbreaking DeepMind AI that mastered the game of Go in just three days using only reinforcement learning and self-play, without human data. Its key innovation is learning solely from the raw board state, abandoning human-precedent for a more efficient, superhuman approach. Reinforcement learning (RL) is a machine learning method where an autonomous agent learns to make optimal, sequential decisions by interacting with an environment through trial and error.

**Results:**

| Dataset | ER-1K | WS-1K | ER-16K | WS-16K | ER-10M | WS-10M | SS-CIR | SS-LP | SS-Web | SS-FE |
|---|---|---|---|---|---|---|---|---|---|---|
| Unordered | 34.3 | 59.2 | 732.8 | 265.35 | 42923 | 16415 | 4.2 | 4.25 | 3.75 | 4.85 |
| Ordered | 32.45 | 57.35 | 715.2 | 261.8 | 40347 | 15922 | 3.15 | **2.95** | 2.6 | 4.05 |
| Dynamic | 32.2 | 57.15 | 708.5 | 261.2 | 37524 | 15843 | 3.55 | 3.15 | 2.7 | 4.25 |
| FCN-train | **29.58** | **52.5** | **660.19** | **237.03** | **35362** | **14924** | **3.0** | **2.95** | **2.4** | **3.75** |
| FCN-test | 31.7 | 56.59 | 702.57 | 258.3 | 37849 | 15023 | 3.1 | **2.95** | 2.55 | 4.1 |
| FCN-gen | 33.9 | 57.66 | 708.13 | 267.53 | 43415 | 17262 | 4.15 | 4.3 | 3.7 | 4.95 |

## Graph Colouring Meets Deep Learning: Effective Graph Neural Network Models for Combinatorial Problems

**Link:** `https://arxiv.org/pdf/1903.04598`

**Summary:** This article introduces a Graph Neural Network (GNN) framework aimed at solving the decision version of the graph coloring problem (GCP) without requiring prior reductions to other problems like SAT

**Solution & Method:** The solution relies on a message-passing algorithm where two types of nodes, the graph's vertices and the candidate colors, communicate to update their multidimensional embeddings. Trained as a binary classifier to answer whether a graph accepts a specific coloring, it uses an adversarial training strategy utilizing graphs generated on the verge of satisfiability (phase transition). Post-processing extracts constructive color assignments by running a k-means clustering algorithm on the vertex embeddings

**Benchmarks:** Tested on standard instances from the COLOR02/03/04 Workshop dataset and various random graphs (power-law trees, small-world networks). The models and instances: `https://machine-reasoning-ufrgs.github.io/GNN-GCP/`

**Other Info:** Message passing in graphs entails nodes exchanging information (messages) with their neighbors iteratively. This process updates each node's state based on aggregated information, enabling nodes to integrate knowledge from their local neighborhoods. This correlation in graph networks supports tasks such as node classification or link prediction.

**Results:**

| Instance | Size | $\chi$ | Computed $\chi$ | | |
|---|---|---|---|---|---|
| | | | GNN | Tabucol | Greedy |
| queen5_5 | 25 | 5 | 6 | **5** | 8 |
| queen6_6 | 36 | 7 | **7** | 8 | 11 |
| myciel5 | 47 | 6 | 5 | **6** | **6** |
| queen7_7 | 49 | 7 | 8 | 8 | 10 |
| queen8_8 | 64 | 9 | 8 | 10 | 13 |
| 1-Insertions_4 | 67 | 4 | **4** | 5 | 5 |
| huck | 74 | 11 | 8 | **11** | **11** |
| jean | 80 | 10 | 7 | **10** | **10** |
| queen9_9 | 81 | 10 | 9 | 11 | 16 |
| david | 87 | 11 | 9 | **11** | 12 |
| mug88_1 | 88 | 4 | 3 | **4** | **4** |
| myciel6 | 95 | 7 | **7** | **7** | **7** |
| queen8_12 | 96 | 12 | 10 | **12** | 15 |
| games120 | 120 | 9 | 6 | **9** | **9** |
| queen11_11 | 121 | 11 | 12 | NA | 17 |
| anna | 138 | 11 | **11** | **11** | 12 |
| 2-Insertions_4 | 149 | 4 | **4** | 5 | 5 |
| queen13_13 | 169 | 13 | 14 | NA | 21 |
| myciel7 | 191 | 8 | NA | **8** | **8** |
| homer | 561 | 13 | 14 | **13** | 15 |

## Graph Coloring with Physics-Inspired Graph Neural Networks

**Link:** `https://arxiv.org/pdf/2202.01606`

**Summary:** This project utilizes concepts from statistical physics to solve graph coloring as an unsupervised multi-class node classification problem via GNNs

**Solution & Method:** The method strictly relies on an unsupervised training strategy directed by a differentiable loss function based on the anti-ferromagnetic Potts model, a statistical physics concept that naturally applies energy penalties when adjacent nodes share the same color. The GNN (using structures like GCN or GraphSAGE) outputs a soft, probabilistic distribution of color classes, which are then mapped to distinct color assignments using a simple projection heuristic (like argmax)

**Benchmarks:** Benchmarked on standard instances from the COLOR dataset (like Queens and Myciel graphs) and large real-world citation graphs (Cora, Citeseer, Pubmed). A demo: `https://github.com/amazon-research/gcp-with-gnns-example`

**Other Info:** -

**Results:**

| graph | nodes | edges | density | colors $q$ | $\bar{\chi}_{\text{greedy}}$ | $\bar{\chi}_{\text{GNN}}$ | Tabucol | Tabucol [33] | GNN [33] | PI-GCN | PI-SAGE | error $\epsilon$ |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| anna | 138 | 493 | 5.22% | 11 | 11 | 11 | **0** | **0** | 1 | 1 | **0** | 0.00% |
| jean | 80 | 254 | 8.04% | 10 | 10 | 10 | **0** | **0** | **0** | **0** | **0** | 0.00% |
| myciel5 | 47 | 236 | 21.83% | 6 | 6 | 6 | **0** | **0** | **0** | **0** | **0** | 0.00% |
| myciel6 | 95 | 755 | 16.91% | 7 | 7 | 7 | **0** | **0** | **0** | **0** | **0** | 0.00% |
| queen5-5 | 25 | 160 | 53.33% | 5 | 5 | 5 | **0** | **0** | **0** | **0** | **0** | 0.00% |
| queen6-6 | 36 | 290 | 46.03% | 7 | 8 | 7 | **0** | **0** | 4 | 1 | **0** | 0.00% |
| queen7-7 | 49 | 476 | 40.48% | 7 | 9 | 7 | **0** | 10 | 15 | 8 | **0** | 0.00% |
| queen8-8 | 64 | 728 | 36.11% | 9 | 10 | 10 | **0** | 8 | 7 | 6 | 1 | 0.14% |
| queen9-9 | 81 | 1056 | 32.59% | 10 | 12 | 11 | **0** | 5 | 13 | 13 | 1 | 0.09% |
| queen8-12 | 96 | 1368 | 30.00% | 12 | 13 | 12 | **0** | 10 | 7 | 10 | **0** | 0.00% |
| queen11-11 | 121 | 1980 | 27.27% | 11 | 15 | 14 | 20 | 33 | 33 | 37 | **17** | 0.86% |
| queen13-13 | 169 | 3328 | 23.44% | 13 | 17 | 17 | 35 | 42 | 40 | 61 | **26** | 0.78% |

## Rethinking Graph Neural Networks for the Graph Coloring Problem

**Link:** `https://arxiv.org/pdf/2208.06975`

**Summary:** The article diagnoses why popular aggregation-combine GNNs (AC-GNNs) often fail at graph coloring and proposes a simple GNN architecture designed to overcome these mathematical limitations

**Solution & Method:** They propose the Graph Discrimination Network (GDN), which avoids combining the node's ego-information with its neighbors, uses an unsupervised margin loss to ensure connected nodes receive distinct probability distributions, and incorporates customized tensor-based preprocess and postprocess operations. It also proposes a combination with Integer Linear Programming (ILP) solvers to further refine complex results

**Benchmarks:** Evaluated on a real-world Layout dataset from circuit design, Citation datasets (Cora, Citeseer, Pubmed), and the COLOR dataset

**Other Info:** Integer Linear Programming (ILP) is an optimization technique that deals with problems involving linear relationships and integer variables. It extends Linear Programming (LP) by restricting decision variables to integer values. (facut si la AEA, curs+seminar 2)

**Results:**

| **Graph** | $\|\mathcal{V}\|$ | $\|\mathcal{E}\|$ | $d\%$ | $\mathcal{X}(k)$ | **GNN-GCP** | | **Tabucol** | | **HybridEA** | | **GDN** | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | Cost | Time | Cost | Time | Cost | Time | Cost | Time |
| 35158 | 641202 | 787242 | - | 3 | 386009 | 3896 | 2392 | 82301 | 1562 | 133285 | **1557**±45 | 5.84 ±0.23 |
| | | | | | 247.9 | 667.1 | 1.54 | 14092 | 1.00 | 22822 | **1.0** | **1.0** |
| Cora | 2708 | 5429 | 0.15 | 5 | 1291 | 3.90 | 31 | 15410 | 0 | 18921 | **0** ±0 | 0.81 ±0.08 |
| Citeseer | 3327 | 4732 | 0.09 | 6 | 1733 | 2.74 | 6 | 44700 | 0 | 24230 | **0** ±0 | 1.42 ±0.15 |
| Pubmed | 19717 | 44338 | 0.03 | 8 | 4393 | 4.50 | - | >24h | - | >24h | **21** ±4 | 1.41 ±0.12 |
| | | | | | 353.2 | 3.06 | - | - | - | - | **1.0** | **1.0** |
| jean | 80 | 254 | 8 | 10 | 76 | 0.06 | **0** | 0.95 | **0** | 0.01 | **0** ±1 | 0.13 ±0.02 |
| anna | 138 | 493 | 5 | 11 | 87 | 0.08 | **0** | 3.23 | **0** | 0.02 | **0** ±0 | 0.17 ±0.03 |
| huck | 74 | 301 | 11 | 11 | 117 | 0.05 | **0** | 0.15 | **0** | 0.01 | **0** ±0 | 0.06 ±0.02 |
| david | 87 | 406 | 11 | 11 | - | - | **0** | 4.83 | **0** | 0.01 | **0** ±0 | 0.19 ±0.01 |
| homer | 561 | 1628 | 1 | 13 | 1628 | 1.09 | **0** | 274 | **0** | 0.06 | **0** ±1 | 0.29 ±0.02 |
| myciel5 | 47 | 236 | 22 | 6 | 35 | 0.04 | **0** | 0.20 | **0** | 0.01 | **0** ±0 | 0.12 ±0.01 |
| myciel6 | 95 | 755 | 17 | 7 | 94 | 4.33 | **0** | 0.79 | **0** | 0.01 | **0** ±0 | 0.21 ±0.02 |
| games120 | 120 | 638 | 9 | 9 | 301 | 0.07 | **0** | 0.93 | **0** | 0.01 | **0** ±1 | 0.08 ±0.01 |
| Mug88_1 | 88 | 146 | 4 | 3 | 146 | 0.33 | **0** | 0.12 | **0** | 0.01 | **0** ±0 | 0.01 ±0 |
| 1-Insertions_4 | 67 | 232 | 10 | 2 | 42 | 0.05 | **0** | 0.16 | **0** | 0.01 | **0** ±0 | 0.07 ±0 |
| 2-Insertions_4 | 212 | 1621 | 7 | 4 | 360 | 0.09 | **1** | 255 | **1** | 101.1 | **1** ±0 | 60.08 ±0.01 |
| Queen5_5 | 25 | 160 | 53 | 5 | 37 | 0.03 | **0** | 0.13 | **0** | 0.07 | **0** ±0 | 0.05 ±0.01 |
| Queen6_6 | 36 | 290 | 46 | 6 | 290 | 0.38 | **0** | 4.93 | **0** | 1.89 | **0** ±0 | 0.08 ±0 |
| Queen7_7 | 49 | 476 | 40 | 7 | 126 | 0.04 | 10 | 36.9 | **9** | 51.8 | **9** ±1 | 0.38 ±0.08 |
| Queen8_8 | 64 | 728 | 36 | 8 | 188 | 0.05 | 8 | 61.3 | 5 | 74.1 | **2** ±0 | 0.13 ±0.03 |
| Queen9_9 | 81 | 1056 | 33 | 9 | 296 | 0.07 | **5** | 97.8 | 6 | 126.9 | 6 ±1 | 0.09 ±0.01 |
| Queen8_12 | 96 | 1368 | 30 | 12 | 260 | 0.10 | 10 | 139 | 3 | 92.9 | **0** ±0 | 0.58 ±0.09 |
| Queen11_11 | 121 | 3960 | 55 | 11 | 396 | 0.10 | 33 | 213 | 22 | 141.3 | **21** ±3 | 0.07 ±0.01 |
| Queen13_13 | 169 | 6656 | 47 | 13 | 728 | 0.20 | 42 | 401 | 37 | 213 | **33** ±2 | 2.38 ±0.32 |
| | | | | | - | - | 1.51 | 22.63 | 1.15 | 12.10 | **1.0** | **1.0** |

## Generating a Graph Colouring Heuristic with Deep Q-Learning and Graph Neural Networks

**Link:** `https://arxiv.org/pdf/2304.04051`

**Summary:** This project focuses on automatically generating competitive greedy construction heuristics for coloring by framing node selection as a sequential decision-making task

**Solution & Method:** The ReLCol algorithm integrates Deep Q-learning (DQN) with a GNN to sequentially select the best next uncolored vertex. A major innovation is the parameterization of the state as a complete graph; this structural alteration allows the GNN's message-passing mechanism to freely exchange data between all pairs of vertices, not exclusively adjacent ones

**Benchmarks:** The models were trained on randomly generated graphs (including Erdős-Renyi and Barabasi-Albert networks). The tests were performed against the COLOR02 benchmark dataset and Spinrad graphs. The generating algorithms and datasets are available at `https://github.com/gpdwatkins/graph_colouring_with_RL`

**Other Info:** -

**Results:**

| Graph instance | $n$ | $\chi$ | Random | LF | SL | DSATUR | Gianinazzi et al. | ReLCol |
|---|---|---|---|---|---|---|---|---|
| queen5_5 | 25 | 5 | $2.3^{\pm 0.1}$ | 2 | 3 | **0** | $2.1^{\pm 0.3}$ | $0.2^{\pm 0.11}$ |
| queen6_6 | 36 | 7 | $2.4^{\pm 0.06}$ | 2 | 4 | 2 | $3.2^{\pm 0.16}$ | $\mathbf{1}^{\pm 0}$ |
| myciel5 | 47 | 6 | $0.1^{\pm 0.03}$ | 0 | 0 | 0 | $0.1^{\pm 0.08}$ | $0^{\pm 0}$ |
| queen7_7 | 49 | 7 | $4^{\pm 0.06}$ | 5 | 3 | 4 | $3.3^{\pm 0.32}$ | $\mathbf{2.2}^{\pm 0.16}$ |
| queen8_8 | 64 | 9 | $3.4^{\pm 0.07}$ | 4 | 5 | 3 | $3.3^{\pm 0.24}$ | $\mathbf{2.1}^{\pm 0.14}$ |
| 1-Insertions_4 | 67 | 4 | $1.2^{\pm 0.04}$ | 1 | 1 | 1 | $1^{\pm 0}$ | $1^{\pm 0}$ |
| huck | 74 | 11 | $0^{\pm 0}$ | 0 | 0 | 0 | $0^{\pm 0}$ | $0^{\pm 0}$ |
| jean | 80 | 10 | $0.3^{\pm 0.05}$ | 0 | 0 | 0 | $0^{\pm 0}$ | $0.3^{\pm 0.12}$ |
| queen9_9 | 81 | 10 | $3.9^{\pm 0.07}$ | 5 | 5 | 3 | $5^{\pm 0}$ | $\mathbf{2.7}^{\pm 0.25}$ |
| david | 87 | 11 | $0.7^{\pm 0.07}$ | 0 | 0 | 0 | $0.4^{\pm 0.14}$ | $1.2^{\pm 0.33}$ |
| mug88_1 | 88 | 4 | $0.1^{\pm 0.02}$ | 0 | 0 | 0 | $0^{\pm 0}$ | $0^{\pm 0}$ |
| myciel6 | 95 | 7 | $0.3^{\pm 0.05}$ | 0 | 0 | 0 | $0.8^{\pm 0.17}$ | $0^{\pm 0}$ |
| queen8_12 | 96 | 12 | $3.3^{\pm 0.06}$ | 3 | 3 | **2** | $4^{\pm 0}$ | $2.6^{\pm 0.18}$ |
| games120 | 120 | 9 | $0^{\pm 0.01}$ | 0 | 0 | 0 | $0^{\pm 0}$ | $0^{\pm 0}$ |
| queen11_11 | 121 | 11 | $5.9^{\pm 0.07}$ | 6 | 6 | **4** | $6^{\pm 0}$ | $5.4^{\pm 0.25}$ |
| anna | 138 | 11 | $0.2^{\pm 0.04}$ | 0 | 0 | 0 | $0^{\pm 0}$ | $0.4^{\pm 0.18}$ |
| 2-Insertions_4 | 149 | 4 | $1.5^{\pm 0.05}$ | 1 | 1 | 1 | $1^{\pm 0}$ | $1^{\pm 0}$ |
| queen13_13 | 169 | 13 | $6.5^{\pm 0.07}$ | 10 | 9 | **4** | $8^{\pm 0}$ | $6.3^{\pm 0.24}$ |
| myciel7 | 191 | 8 | $0.6^{\pm 0.06}$ | 0 | 0 | 0 | $0.3^{\pm 0.14}$ | $0.2^{\pm 0.11}$ |
| homer | 561 | 13 | $1^{\pm 0.07}$ | 0 | 0 | 0 | $0^{\pm 0}$ | $0.6^{\pm 0.22}$ |
| Average | | | $1.9^{\pm 0.01}$ | 1.95 | 2 | 1.2 | $1.92^{\pm 0.05}$ | $1.35^{\pm 0.05}$ |

## Graph Neural Networks as Ordering Heuristics for Parallel Graph Coloring

**Link:** https://arxiv.org/pdf/2408.05054

**Summary:** This paper proposes a highly optimized, shallow GNN-based ordering heuristic built explicitly to compete with the fastest non-trivial greedy heuristics in execution speed, solution quality, and parallel scalability

**Solution & Method:** The method utilizes a very shallow GraphSAGE model (only 2 to 4 layers) to directly predict node priority values, constructing a partial ordering. This generated ordering is subsequently processed using the parallel Jones-Plassmann (JP) algorithm to color the graph concurrently. Training occurs in two stages: supervised learning on fast established heuristics (like Smallest Degree Last), followed by population-based genetic training to directly mutate model parameters and refine coloring numbers

**Benchmarks:** Evaluated on 67 graphs from the Stanford Large Network Dataset Collection (SNAP) and 80 instances from the DIMACS implementation challenge. Both the source code (written in highly optimized C) and trained models are open-source at https://github.com/KennethLangedal/GNN-COLORING

**Other Info:** Ordering heuristics are rules of thumb used in algorithms, particularly backtracking searches for Constraint Satisfaction Problems (CSPs), to determine the sequence in which variables are assigned or values are tried. They aim to minimize search space and find solutions faster by prioritizing choices that are most likely to fail early ("first-fail") or by utilizing domain-specific information, such as the Least Constrained Variable or Minimum Remaining Values (MRV).

**Results:** -

---

## Solving the graph coloring problem via hybrid genetic algorithms

**Link:** https://www.sciencedirect.com/science/article/pii/S1018363913000135

**Summary:** The paper proposes a method that uses evolutionary search (genetic algorithms) to explore possible colorings and a local optimization heuristic (DBG) to fix conflicts, producing efficient solutions for the graph coloring problem.

**Solution & Method:** The genetic algorithm (GA) explores the solution space using evolutionary principles. The GA performs global exploration of the search space. To improve solutions, the algorithm integrates a local search heuristic called DBG. The DBG procedure: examines vertices causing conflicts, attempts to reassign colors locally, reduces the number of color conflicts, moves the solution toward a valid coloring. This step performs local optimization. This hybridization allows the method to: escape local minima, improve solution quality, converge faster.

**Benchmarks:** The algorithm is tested on DIMACS benchmark graphs, and the results show that the proposed hybrid method produces competitive and effective solutions compared with other algorithms for graph coloring.(Graph Coloring Instances )

**Other Info:** -

**Results:**

**Table 1** Experimental results.

| Graph | $\|V\|$ | $\|E\|$ | $k^*$ | $k_{Ours}$ | $T_{avg}(s)$ | succ |
|---|---|---|---|---|---|---|
| anna | 138 | 493 | 11 | 11 | 11.63 | 12/15 |
| david | 87 | 406 | 11 | 11 | 10.89 | 14/15 |
| huck | 74 | 301 | 11 | 11 | 11.10 | 15/15 |
| 2-Inser-3 | 37 | 72 | 4 | 4 | 2.23 | 15/15 |
| 3-Inser-3 | 56 | 110 | 4 | 4 | 3.37 | 15/15 |
| jean | 80 | 254 | 10 | 10 | 9.92 | 15/15 |
| queen5.5 | 25 | 160 | 5 | 5 | 1.20 | 15/15 |
| queen6.6 | 36 | 290 | 7 | 7 | 1.32 | 15/15 |
| queen7.7 | 49 | 476 | 7 | 7 | 6.91 | 15/15 |
| queen8.8 | 64 | 728 | 9 | 9 | 9.87 | 11/15 |
| queen9.9 | 81 | 1056 | 10 | 10 | 13.96 | 13/15 |
| miles250 | 128 | 387 | 8 | 8 | 4.39 | 11/15 |
| miles500 | 128 | 1170 | 20 | 20 | 14.48 | 10/15 |
| games 120 | 120 | 638 | 9 | 9 | 10.77 | 15/15 |
| mug88-1 | 88 | 146 | 4 | 4 | 4.05 | 15/15 |
| mug88-25 | 88 | 146 | 4 | 4 | 3.67 | 13/15 |
| mug 100-1 | 100 | 166 | 4 | 4 | 8.34 | 11/15 |
| mug 100-25 | 100 | 166 | 4 | 4 | 8.21 | 13/15 |
| mycie13 | 11 | 20 | 4 | 4 | 0.01 | 15/15 |
| mycie14 | 23 | 71 | 5 | 5 | 0.89 | 15/15 |
| mycie15 | 47 | 236 | 6 | 6 | 5.41 | 15/15 |
| mycie16 | 95 | 755 | 7 | 7 | 12.77 | 11/15 |
| mycie17 | 191 | 2360 | 8 | 8 | 20.19 | 9/15 |
| dsjc 125.1 | 125 | 736 | 5 | 6 | 13.12 | 14/15 |
| dsjc1 125.5 | 125 | 3891 | 17 | 17 | 19.71 | 8/15 |
| dsjc1 125.9 | 125 | 6961 | 44 | 44 | 35.87 | 7/15 |
| dsjc1250.1 | 250 | 3218 | 8 | 8 | 26.07 | 10/15 |
| dsjc1250.9 | 250 | 27897 | 72 | 72 | 75.81 | 6/15 |
| fpso12.i.1 | 496 | 11654 | 65 | 65 | 82.03 | 2/15 |
| fpso12.i.2 | 451 | 8691 | 30 | 30 | 69.79 | 6/15 |
| fpso12.i.3 | 425 | 8688 | 30 | 30 | 67.14 | 4/15 |
| zeroin.i.1 | 211 | 4100 | 49 | 49 | 28.24 | 5/15 |
| zeroin.i.2 | 211 | 3541 | 30 | 30 | 83.39 | 9/15 |
| zeroin.i.3 | 206 | 3540 | 30 | 30 | 27.06 | 6/15 |
| mulsol.i.1 | 197 | 3925 | 49 | 49 | 25.75 | 2/15 |
| mulsol.i.2 | 188 | 3885 | 31 | 31 | 22.07 | 7/15 |
| mulsol.i.3 | 184 | 3916 | 31 | 31 | 23.49 | 5/15 |
| mulsol.i.4 | 185 | 3946 | 31 | 31 | 25.22 | 4/15 |
| mulsol.i.5 | 186 | 3973 | 31 | 31 | 28.33 | 7/15 |

**Table 2** Experimental results.

| Graph | $\|V\|$ | Wu and Hao (2012) | Lu and Hao (2010) | Blochliger and Zufferey (2008) | Hertz et al. (2008) | Malaguti and Monaci (2008) | Chiarandini and Stutzle (2002) | $k_{Ours}$ | $T_{avg}(s)$ | succ |
|---|---|---|---|---|---|---|---|---|---|---|
| dsjc250.5 | 250 | * | **28** | * | * | **28** | **28** | **28** | 89 | 18/20 |
| dsjc500.1 | 500 | * | **12** | **12** | **12** | **12** | **12** | **12** | 112 | 15/20 |
| dsjc500.5 | 500 | * | **48** | **48** | **48** | **48** | 49 | **48** | 147 | 18/20 |
| dsjc500.9 | 500 | * | **126** | 127 | **126** | 127 | **126** | **126** | 190 | 19/20 |
| dsjc 1000.1 | 1000 | **20** | **20** | **20** | **20** | **20** | * | **20** | 4982 | 9/20 |
| dsjc1000.5 | 1000 | **83** | **83** | 89 | 86 | **83** | 89 | **83** | 2164 | 13/20 |
| dsjc1000.9 | 1000 | **222** | 223 | 226 | 224 | 224 | * | 224 | 8092 | 7/20 |
| dsjr500.1c | 500 | * | **85** | **85** | **85** | **85** | * | **85** | 581 | 16/20 |
| dsjr500.5 | 500 | * | **122** | 125 | 125 | **122** | 124 | 124 | 728 | 13/20 |
| r250.5 | 250 | * | **65** | 66 | * | **65** | * | **65** | 261 | 17/20 |
| r250.1 | 250 | * | * | * | * | **8** | * | **8** | 308 | 20/20 |
| r250.1c | 250 | * | * | **64** | * | **64** | * | **64** | 266 | 18/20 |
| r1000.1 | 1000 | * | * | * | * | **20** | * | **20** | 895 | 10/20 |
| r1000.1c | 1000 | 101 | **98** | **98** | * | **98** | * | **98** | 1329 | 6/20 |
| r1000.5 | 1000 | 249 | 245 | 248 | * | **234** | * | 242 | 1507 | 3/20 |
| le450_15*c* | 450 | * | **15** | **15** | **15** | **15** | **15** | **15** | 93 | 20/20 |
| le450_15*d* | 450 | * | 15 | 15 | 15 | 15 | 15 | 15 | 228 | 20/20 |
| le450_25*c* | 450 | * | **25** | **25** | **25** | **25** | 26 | **25** | 740 | 20/20 |
| le450_25*d* | 450 | * | 26 | 25 | 25 | 25 | 26 | 25 | 382 | 18/20 |
| flat300_20_0 | 300 | * | * | * | * | **20** | * | **20** | 241 | 20/20 |
| flat300_26_0 | 300 | * | **26** | * | * | **26** | **26** | **26** | 275 | 16/20 |
| flat300_28_0 | 300 | * | 29 | **28** | **28** | 31 | 31 | **28** | 319 | 18/20 |
| flat1000_50_0 | 1000 | **50** | **50** | **50** | **50** | **50** | * | **50** | 802 | 9/20 |
| flat1000_60_0 | 1000 | **60** | **60** | **60** | **60** | **60** | * | **60** | 1344 | 5/20 |
| flat1000_76_0 | 1000 | **82** | **82** | 87 | 85 | **82** | * | **82** | 3795 | 7/20 |
| C2000.5 | 2000 | **146** | 148 | * | * | * | * | 151 | 8421 | 8/20 |
| C2000.9 | 2000 | **409** | * | * | * | * | * | 411 | 7368 | 5/20 |
| C4000.5 | 4000 | **260** | 272 | * | * | * | * | 282 | 212881 | 2/20 |
| latin_*sqr*_10 | 900 | * | **99** | * | * | 101 | **99** | **99** | 1267 | 11/20 |

# 1 Benchmark instances

`https://mat.tepper.cmu.edu/COLOR/instances.html`

DSJ: random graphs and DSJC are standard (n,p) random graphs. DSJR are geometric graphs, with DSJR..c being complements of geometric graphs.

CUL: Quasi-random coloring problem.

REG: Problem based on register allocation for variables in real codes.

LEI: Leighton graphs with guaranteed coloring size.

SCH: Class scheduling graphs, with and without study halls.

LAT: Latin square problem.

SGB: Graphs from Donald Knuth's Stanford GraphBase. These can be divided into:

1. Book Graphs. Given a work of literature, a graph is created where each node represents a character. Two nodes are connected by an edge if the corresponding characters encounter each other in the book. Knuth creates the graphs for five classic works: Tolstoy's Anna Karenina (anna), Dicken's David Copperfield (david), Homer's Iliad (homer), Twain's Huckleberry Finn (huck), and Hugo's Les Misérables (jean).

2. Game Graphs. A graph representing the games played in a college football season can be represented by a graph where the nodes represent each college team. Two teams are connected by an edge if they played each other during the season. Knuth gives the graph for the 1990 college football season.

3. Miles Graphs. These graphs are similar to geometric graphs in that nodes are placed in space with two nodes connected if they are close enough. These graphs, however, are not random. The nodes represent a set of United States cities and the distance between them is given by by road mileage from 1947. These graphs are also due to Kuth.

4. Queen Graphs. Given an n by n chessboard, a queen graph is a graph on $n^2$ nodes, each corresponding to a square of the board. Two nodes are connected by an edge if the corresponding squares are in the same row, column, or diagonal. Unlike some of the other graphs, the coloring problem on this graph has a natural interpretation: Given such a chessboard, is it possible to place n sets of n queens on the board so that no two queens of the same set are in the same row, column, or diagonal? The answer is yes if and only if the graph has coloring number n. Martin Gardner states without proof that this is the case if and only if n is not divisible by either 2 or 3. In all cases, the maximum clique in the graph is no more than n, and the coloring value is no less than n.

MYC: Graphs based on the Mycielski transformation. These graphs are difficult to solve because they are triangle free (clique number 2) but the coloring number increases in problem size.

COLOR02/03/04: Graph Coloring and its Generalizations `https://mat.tepper.cmu.edu/COLOR02/`

DIMACS Graphs: Benchmark Instances and Best Upper Bounds `https://cedric.cnam.fr/~porumbed/graphs/`

the result tables are now split, using a classification already proposed in certain papers:

(i) local search algorithms (no populations, no hybridisations, no independent set isolation) and

(ii) population-based algorithms, complex hybrid EAs, etc.

| Graph | best K / $\chi$ | References | Graph | best K / $\chi$ | References | Graph | best K / $\chi$ | References |
|---|---|---|---|---|---|---|---|---|
| dsjc250.5 | 28/? | [2,3,7,15,18] | r250.5 | 66/65 | [1] | flat300_28_0 | 28/28 | [1,15,18] |
| dsjc500.1 | 12/? | [1,2,15,18] | r1000.1c | 98/? | [1,18] | flat1000_50_0 | 50/50 | [1,15] |
| dsjc500.5 | 48/? | [1,15,18] | r1000.5 | 234/234 | [13] | flat1000_60_0 | 60/60 | [1,15] |
| dsjc500.9 | 126/? | [1,15,18] | dsjr500.1c | 84/84 | [17] | flat1000_76_0 | 85/76 | [18] |
| dsjc1000.1 | 20/? | [1,14,15,18] | dsjr500.5 | 122/122 | [13] | latin_square | 99/? | [2] |
| dsjc1000.5 | 85/? | [18] | le450_25c | 25/25 | [1,18] | C2000.5 | /? | - |
| dsjc1000.9 | 223/? | [18] | le450.25d | 25/25 | [1,18] | C4000.5 | /? | - |

**Hybrid and population-based methods**

| Graph | best K / $\chi$ | References |
|---|---|---|
| dsjc250.5 | 28/? | [6,7,12,14,18,19,20,22] |
| dsjc500.1 | 12/? | [7,11,12,14,19,20,22] |
| dsjc500.5 | 48/? | [6,7,11,14,19,20,22,23] |
| dsjc500.9 | 126/? | [7,11,12,18,19,20,22,23] |
| dsjc1000.1 | 20/? | [6,7,11,14,19,20,21,22,23] |
| dsjc1000.5 | 83/? | [6,11,14,19,20,21,22,23] |
| dsjc1000.9 | 222/? | [21,22,23] |

| Graph | best K / $\chi$ | References |
|---|---|---|
| r250.5 | 65/65 | [12,14,19,20,22,23] |
| r1000.1c | 98/? | [12,14 19,20,22 ] |
| r1000.5 | 234/234 | [14] |
| dsjr500.1c | 85/84 | [12,14,19,20,22] |
| dsjr500.5 | 122/122 | [14,19,20,22,23] |
| le450_25c | 25/25 | [12[p. 353],14,19,20,22,23] |
| le450_25d | 25/25 | [12[pp. 353],14, 19,20,22,23] |

| Graph | best K / $\chi$ | References |
|---|---|---|
| flat300_28_0 | 29/28 | [19,20] |
| flat1000_50_0 | 50/50 | [12,7,14,19,20,21] |
| flat1000_60_0 | 60/60 | [12,7,14,15,19,20,21] |
| flat1000_76_0 | 82/76 | [14,19,20,21,22,23] |
| latin_square | 97/? | [23] |
| C2000.5 | 146/? | [21[1]] |
| C4000.5 | 260/? | [21[1]] |

## 2 Conclusion for the articles we found

Traditional heuristics such as DSATUR and Tabucol remain strong baselines, consistently producing optimal or near-optimal colorings on DIMACS benchmark graphs. In many instances, Tabucol achieves the exact chromatic number, outperforming simple greedy methods.

More recent approaches based on Graph Neural Networks (GNNs), including models such as PI-GCN and PI-SAGE, demonstrate competitive performance, achieving low error rates and accurate color assignments on small and medium graphs. However, these models sometimes still lag behind well-tuned heuristic methods.

Other works propose hybrid or evolutionary algorithms, which combine global search with local optimization to improve solution quality and convergence speed. Additionally, deep learning approaches such as Fully Convolutional Networks (FCN) show promising results on large synthetic graph datasets, outperforming several classical ordering strategies.

Overall, while learning-based methods are becoming increasingly effective, metaheuristic and tabu-search based algorithms remain among the most reliable techniques for obtaining optimal graph colorings.

# Explanation of Graph Coloring Methods: GNN and Tabucol

## 3 Graph Neural Network (GNN) Method

### 3.1 What it is

- The GNN method leverages a **2-layer Graph Convolutional Network (GCN)** designed to produce low-dimensional embeddings for each node in a graph.
- Rather than predicting colors directly, **it learns to map the graph's structural topology into a semantic embedding space**.
- The training is entirely **self-supervised**, meaning it does not require pre-colored graphs as ground truth data. Instead, it relies on a custom **geometric loss function** that actively pulls non-adjacent nodes closer together (encouraging same colors) while pushing adjacent nodes apart by a unified margin (forcing different colors).
- **Once the network generates these spatial embeddings, a downstream processing pipeline uses these continuous coordinate representations to cluster the nodes and safely allocate integer colors, essentially reducing the combinatorial coloring problem into geometric distance clustering.**

### 3.2 Iterations

- **GNN Training (Epochs):** To avoid unnecessary computation and prevent overfitting, **the system scales** the learning epochs dynamically based on the topological size of the graph.
- **Iterated Local Search (ILS):** After the initial color alignment, a subsequent refinement phase (Iterated Local Search) runs for up to **500 iterations**. During this loop, the algorithm aggressively targets the rarest color class in the graph and attempts to dissolve it entirely by reassigning its nodes to other legal colors.

### 3.3 Optimizations Applied and Why

- **Vectorized Hinge Loss over Edges:**
  - ***What:*** The backpropagation computes the hinge loss for every single edge simultaneously without looping through individual node pairs. It also samples negative random subsets (non-edges) identically.
  - ***Why:*** Iterating node-to-node would choke the computation time. Vectorizing the cost over the adjacency matrix provides massive continuous chunks of arithmetic for the process scheduler.

## 4 TabuCol Method

### 4.1 What it is

- TabuCol is an application of **Tabu Search, a highly successful metaheuristic algorithm** tailored specifically for the vertex coloring problem.
- The algorithm continually tries to discover a **0-conflict state utilizing exactly $k$ colors.**

- **The process initiates with a random assignment across $k$ colors. Naturally, this generates numerous edge conflicts (where adjacent nodes erroneously share identical colors).**
- TabuCol iteratively **cycles through conflicting nodes, flipping their colors to locally minimize the immediate number of conflicts.**
- To evade infinite loops where the algorithm alternates back and forth between two identical states (getting trapped in local optima), a **short-term memory mechanism called the "Tabu List" records flipped colors and forbids a node from accepting a recently discarded color for a certain duration.**

### 4.2 Iterations

- **Core Loop Processing:** Every attempt to format the graph with $k$ colors runs for up to **50,000 iterations** without halting. **If 0 conflicts are reached earlier, it terminates immediately.**
- **Independent Restarts:** If the iterations are exhausted and conflicts remain, the algorithm automatically enacts up to **3 totally random independent restarts on the exact same $k$ before officially declaring it unsolvable.**
- **Sequential Optimization Outer Loop:** The architecture progressively decrements $k$ (seeded from a rapid greedy theoretical upper limit bound estimation), executing the iterations at each target $k$ until it can no longer compress the chromatic number.

### 4.3 Optimizations Applied and Why

- **Dynamic Tabu Tenure Formula:**
  - ***What:*** The lifespan of how long a color remains forbidden ("tabu") is dynamically tied to the number of colors $k$ (explicitly targeting a recommended factor $0.6 \times k$).
  - ***Why:*** Establishing a stagnant integer for the tabu list yields poor results.
- $\mathcal{O}(1)$ **Adjacency Color Lookup Map:**
  - ***What:*** TabuCol utilizes a state grid (`adj_color_table`) tracking the precise quantity of surrounding colors attached to every localized node.
  - ***Why:*** Traditionally, deducing neighbor conflict frequencies imposes $\mathcal{O}$(Neighbor-Degree) time complexity. Establishing a **pre-maintained matrix simplifies every query to instant $\mathcal{O}(1)$ memory lookups**, which heavily compresses processing time given the operation functions millions of times per run.
- $\mathcal{O}(1)$ **Live Conflict Isolation Array:**
  - ***What:*** Contains an exclusively managed active array map dictating exactly which targeted nodes are participating in unresolved conflicts.
  - ***Why:*** In each iteration, TabuCol uniformely selects a random node that is in a conflict natively. Without an actively managed subset logic, the entire matrix space would be iterated every tick to manually discern conflict participations. Structuring index arrays directly slashes the operation cost drastically.

# Graph Coloring Benchmark Results

## Set1-myciel

| Instance | Nodes | Edges | GNN mean k | GNN min k | GNN max k | GNN stdev | GNN time (s) | Tabu mean k | Tabu min k | Tabu max k | Tabu stdev | Tabu time (s) | Best k |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| myciel3 | 11 | 20 | 4.00 | 4 | 4 | 0.00 | 0.0157 | 4.00 | 4 | 4 | 0.00 | 0.2864 | 4 |
| myciel4 | 23 | 71 | 5.00 | 5 | 5 | 0.00 | 0.0243 | 5.00 | 5 | 5 | 0.00 | 0.4291 | 5 |
| myciel5 | 47 | 236 | 6.00 | 6 | 6 | 0.00 | 0.0528 | 6.00 | 6 | 6 | 0.00 | 0.6091 | 6 |
| myciel6 | 95 | 755 | 7.00 | 7 | 7 | 0.00 | 0.1830 | 7.00 | 7 | 7 | 0.00 | 0.9516 | 7 |
| myciel7 | 191 | 2360 | 8.00 | 8 | 8 | 0.00 | 0.5788 | 8.00 | 8 | 8 | 0.00 | 1.4826 | 8 |

## Set2-queen

| Instance | Nodes | Edges | GNN mean k | GNN min k | GNN max k | GNN stdev | GNN time (s) | Tabu mean k | Tabu min k | Tabu max k | Tabu stdev | Tabu time (s) | Best k |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| queen5_5 | 25 | 160 | 6.13 | 5 | 8 | 1.26 | 0.0384 | 5.33 | 5 | 6 | 0.47 | 0.3026 | 5 |
| queen6_6 | 36 | 290 | 10.00 | 10 | 10 | 0.00 | 0.0601 | 7.73 | 7 | 8 | 0.44 | 0.7252 | 7 |
| queen7_7 | 49 | 476 | 12.53 | 11 | 15 | 1.26 | 0.0890 | 8.00 | 8 | 8 | 0.00 | 0.8997 | 7 |

## Set3

| Instance | Nodes | Edges | GNN mean k | GNN min k | GNN max k | GNN stdev | GNN time (s) | Tabu mean k | Tabu min k | Tabu max k | Tabu stdev | Tabu time (s) | Best k |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| anna | 138 | 493 | 11.00 | 11 | 11 | 0.00 | 0.1299 | 11.00 | 11 | 11 | 0.00 | 1.4365 | 11 |
| david | 87 | 406 | 11.00 | 11 | 11 | 0.00 | 0.0918 | 11.00 | 11 | 11 | 0.00 | 1.2292 | 11 |
| jean | 80 | 254 | 10.00 | 10 | 10 | 0.00 | 0.0648 | 10.00 | 10 | 10 | 0.00 | 0.6908 | 10 |
| huck | 74 | 301 | 11.00 | 11 | 11 | 0.00 | 0.0686 | 11.00 | 11 | 11 | 0.00 | 0.6434 | 11 |
| homer | 561 | 1629 | 13.00 | 13 | 13 | 0.00 | 0.2577 | 13.00 | 13 | 13 | 0.00 | 1.1747 | 13 |
| games120 | 120 | 638 | 9.13 | 9 | 10 | 0.34 | 0.1453 | 9.00 | 9 | 9 | 0.00 | 0.6182 | 9 |

## Set4-medium

| Instance | Nodes | Edges | GNN mean k | GNN min k | GNN max k | GNN stdev | GNN time (s) | Tabu mean k | Tabu min k | Tabu max k | Tabu stdev | Tabu time (s) | Best k |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| flat300_28_0 | 300 | 21695 | 47.27 | 45 | 49 | 1.06 | 4.2284 | 35.80 | 35 | 36 | 0.40 | 8.7718 | 28 |
| dsjc250.5 | 250 | 15668 | 42.13 | 41 | 44 | 0.81 | 2.2571 | 32.13 | 32 | 33 | 0.34 | 6.2456 | 28 |
| le450_25c | 450 | 17343 | 33.07 | 32 | 34 | 0.57 | 3.0228 | 29.00 | 29 | 29 | 0.00 | 3.9931 | 25 |

## Set5-miles

| Instance | Nodes | Edges | GNN mean k | GNN min k | GNN max k | GNN stdev | GNN time (s) | Tabu mean k | Tabu min k | Tabu max k | Tabu stdev | Tabu time (s) | Best k |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| miles250 | 128 | 387 | 8.33 | 8 | 9 | 0.47 | 0.0949 | 8.00 | 8 | 8 | 0.00 | 0.6660 | 8 |
| miles500 | 128 | 1170 | 20.07 | 20 | 21 | 0.25 | 0.2361 | 20.00 | 20 | 20 | 0.00 | 1.3744 | 20 |
| miles750 | 128 | 2113 | 32.00 | 32 | 32 | 0.00 | 0.3967 | 31.33 | 31 | 32 | 0.47 | 1.2297 | 31 |
| miles1000 | 128 | 3216 | 44.60 | 44 | 45 | 0.49 | 0.5896 | 42.67 | 42 | 43 | 0.47 | 2.1177 | 42 |
| miles1500 | 128 | 5198 | 73.00 | 73 | 73 | 0.00 | 0.9148 | 73.00 | 73 | 73 | 0.00 | 3.8054 | 73 |

## Set6-large

| Instance | Nodes | Edges | GNN mean k | GNN min k | GNN max k | GNN stdev | GNN time (s) | Tabu mean k | Tabu min k | Tabu max k | Tabu stdev | Tabu time (s) | Best k |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| dsjc500.9 | 500 | 112437 | 177.47 | 174 | 181 | 1.86 | 13.0093 | 139.47 | 138 | 141 | 0.81 | 54.0298 | 126 |
| dsjc1000.5 | 1000 | 249826 | 131.33 | 130 | 135 | 1.35 | 40.3504 | 102.67 | 101 | 104 | 0.70 | 63.5095 | 85 |